Analysis of Massive Data Sets

http://www.fer.hr/predmet/avsp

Prof. dr. sc. Siniša Srbljić

Izv. prof. dr. sc. Marin Šilić

Izv. prof. dr. sc. Goran Delač

Izv. prof. dr. sc. Klemo Vladimir

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Copyright © 2022 CCL : Analysis of massive data sets

Mining Data streams 1

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Mining Data streams

- Outline
  1. Introduction/Stream Data Model
  2. Sampling streams (*Random sample/Sliding windows*)
  3. Filtering streams (*Bloom filter*)
  4. Counting distinct elements (*Flajolet-Martin*)
  5. *Estimating moments (Alon-Matias-Szegedy)*
  6. *Counting 1's (Datar-Gionis-Indyk-Motwani)*
  7. *Literature*

Copyright © 2022 CCL : Analysis of massive data sets

The Stream Data Model

- Data arrives in *streams*
  - It must be processed immediately or stored
  - If not, then it is lost forever
- Data arrives *rapidly*
  - It is not feasible to store it all
- *Infinite* and *non-stationary* data
  - Controlled externally
    - Google queries, Twitter statuses...

Copyright © 2022 CCL : Analysis of massive data sets

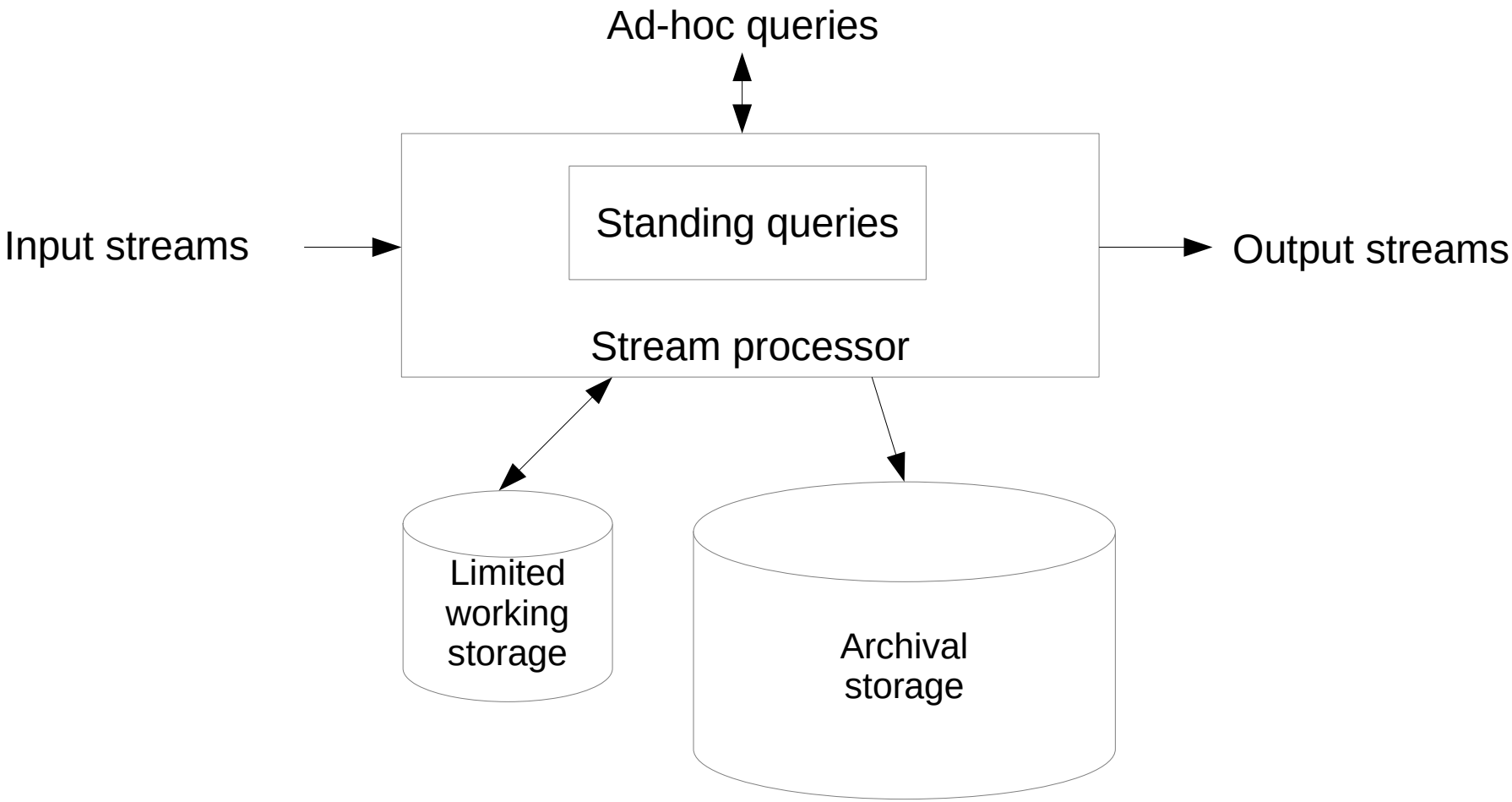
The Stream Data Model

- Examples
  - Web traffic
    - Google search queries
    - Twitter public stream
  - Internet traffic
    - Routing IP packets
  - Sensor data
    - Data rate * Total number of sensors
  - Image data
    - Satellites, surveillance cameras

Copyright © 2022 CCL : Analysis of massive data sets

The Stream Data Model

- Data stream management system



The Stream Data Model

- Data stream management system
  - Analogy to DBMS
  - Archival storage
    - Not used to answer queries
  - Working storage
    - Limited – cannot store all the data from all the streams
    - Stores summarizes or parts of streams
    - Disk or main memory

Copyright © 2022 CCL : Analysis of massive data sets

The Stream Data Model

- Standing queries
  - Execute “permanently”
  - Example: temperature sensor
    - Alert on *x* degrees C
      - Depends only on most recent reading
    - Max temperature
      - Store current maximum
    - Average of *n* recent readings
      - First n readings
        - avg = sum/n
      - Next readings
        - x – new element, y – oldest element
        - new_avg = avg + (x – y)/n

The Stream Data Model

- Ad-hoc queries
  - Asked once about the current state of stream
  - Depend on stream summarization stored in working storage
    - *Sliding windows*
      - Store most recent elements of a stream
      - Stream management system keeps the window fresh
      - Removes oldest elements as new ones come in
    - *Random sample*
      - Store random, but representative sample of a stream

The Stream Data Model

- Online algorithms/learning *
  - In Machine Learning
  - Algorithm slowly adapts to the changes in data
    - For every sample in the data, the model is slightly updated
    - SVM, Perceptron ...

The Stream Data Model

- Conditions
  - Large number of streams
  - Stream elements enter at rapid rate
  - System cannot store the entire streams
- Challenge
  - *How to derive information from the stream using a limited amount of memory?*
- Accuracy/performance trade-off
  - It is much more efficient to get an *approximate* answer than an exact solution

2. Sampling streams

Copyright © 2022 CCL : Analysis of massive data sets

Sampling streams

- Motivating example: Search engine queries
  - Elements are tuples
    - (user, query, timestamp)
  - Need to study the behavior of typical users
    - Find *fraction of unique queries* in the past month
  - Sample 10% of the whole stream

Copyright © 2022 CCL : Analysis of massive data sets

Sampling streams

- Motivating example: Search engine queries
  - Obvious approach for 10% sample
    - For each search query, generate a random number in range 0 – 9
    - Store the query if number is 0
    - On average, 10% of the queries for each user after some time
      - the law of large numbers

Sampling streams

- Motivating example: Search engine queries
  - Obvious approach
    - What is the problem with this approach?
      - The fraction of **unique queries** in the sample will not be the fraction for the stream as a whole
      - Probability of a given query appearing to be unique in the sample is distorted by the sample
      - Example
        - Query appeared exactly two times in the whole stream
        - System sampled only first occurrence of the query
        - Query appears unique in the sample, but it is not unique in the stream

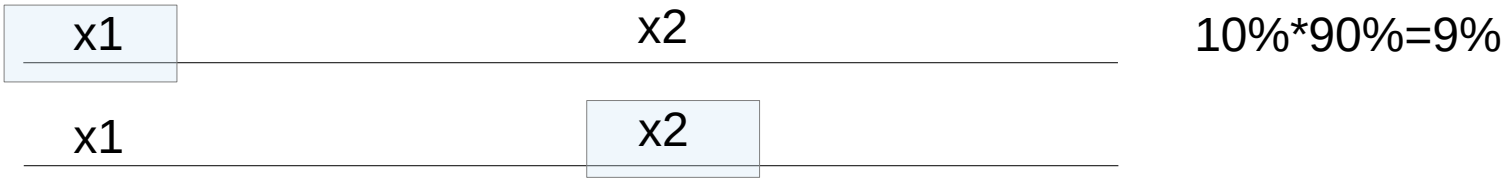
Copyright © 2022 CCL : Analysis of massive data sets

Sampling streams

- Motivating example: Search engine queries
  - Obvious approach: Random numbers 0-9
    - Finding unique queries
    - Suppose a query is unique
      - Query appears once in the whole stream



      - 10% chance of being in the sample
    - Suppose a query appears twice in the whole stream



Total: **18%** chance of appearing **once** in the sample (looking unique)

Sampling streams

- Motivating example: Search engine queries
  - Obvious approach: Random numbers 0-9
    - **Overestimate** of the true fraction of unique queries
  - Problem
    - Independent selection
    - Each time query arrives, random number is generated
    - Based on query **position** in the stream
  - Solution
    - Sampling by **value**
    - Pick users, not searches

Copyright © 2022 CCL : Analysis of massive data sets

Sampling streams

- **Representative sample**
  - Sample by value: (user, query, t)
  - Keep list of all users with the membership information
    - if user is in the sample (True/False)
  - Procedure when new query arrives
    - User lookup
    - If membership and user is in the sample
      - add this query to the sample
    - If no membership yet
      - Generate random integer between 0 – 9
      - If 0: mark user as member of sample, else mark as not member
    - Problems
      - Not enough memory to keep the list of users
      - Expensive lookup for every query

Copyright © 2022 CCL : Analysis of massive data sets

Sampling streams

- Representative sample (hashing)
  - Sample by value: (user, query, t)
  - **b** = number of users
  - **a/b** = fraction of the users to store in the sample
  - Hash users to **b** buckets (**0** – **b-1**)
    - Hashing as pseudobucketing
  - Store query in the sample if it is hashed to bucket with value less than **a**
    - All or none of specific user's queries are selected
    - Fraction of unique queries is the same as for the stream as a whole

Copyright © 2022 CCL : Analysis of massive data sets

Sampling streams

- Representative sample
  - Sample size
    - What if the total sample size is limited?
  - Bucket management
    - Use more buckets
    - Dynamically adjust the set of buckets
      - If sample gets too large:
        - Pick one bucket that is included in the sample
        - Delete elements from the selected bucket

Sampling streams

- Representative sample
  - Bucket management
    - Hash to 100 buckets
    - Sample: buckets 0 – 9
    - If the sample gets too big
      - get rid of bucket 9
      - 8, 7, …
  - Generalization: sampling key-value pairs
    - Stream elements are tuples
    - Sample is based on picking some set of keys

Copyright © 2022 CCL : Analysis of massive data sets

3. Filtering streams

Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Motivating example
  - Simple spam filter based on email addresses
  - Set S of 1 billion allowed email addresses
  - Not spam, if an email address is a member of S
- How to check membership of S?
  - Hash table
    - Naive solution
    - Works only if all of S can be stored in memory

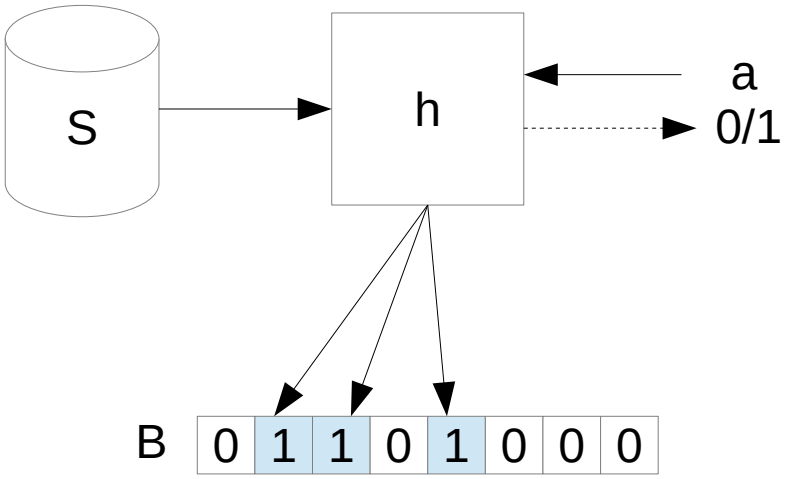
Filtering streams

- **Bloom filter**
  - A Bloom filter placed on the stream of emails will answer with:
    - Not spam
      - Email address is member of S
    - Spam
      - Email address is not member of S
  - Reduced storage costs
  - Bloom filter can have false positives
    - And no false negatives

Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Bloom filter (simple)
  - Create a bit array **B** of **n** bits, initially all zeroes
  - Choose a hash function **h** with range **[0,n-1]**
  - Hash members of **S**, **h(s)**, and set bit **h(s)** to **1**
    - **B[h(s)] = 1**
  - Filtering
    - Emit **a** if **B[h(a)] = 1**



Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Bloom filter is:
  - An **array of n bits**, initially all 0's
  - A **collection of hash functions**
    - $h_1, h_2, \ldots, h_k$
    - Each hash function maps "key" values to **n** buckets, corresponding to the **n** bits of the bit-array
  - A **set S** of **m** key values.

Filtering streams

- Bloom filter
  - The purpose of the Bloom filter
    - allow through all stream elements whose keys are in S, while rejecting *most* of the stream elements whose keys are not in S
  - Procedure
    - Take each key value in S and hash it using each of the k hash functions
    - For some hash function h_i and some key value K in S
      - Set to 1 each bit that is $h_i(K)$

Filtering streams

- Bloom filter: Example
  - Stream elements: words
  - n=10 bits for the Bloom filter
  - Two hash functions:
    - $h_1(x)$ = len(x)%10
    - $h_2(x)$ = ord(x[0])%10

Filtering streams

- Bloom filter: Example
  - Input: “bloom filter”
  - Filter creation:
    - B = 0000000000
    - h_1(“bloom”) = 5, h_2(“bloom”) = 98%10=8
    - B = 0000**1**00**1**0
    - h_1(“filter”) = 6, h_2(“filter”) = 102%10=2
    - B = 00**1**00**11**0**1**0

Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Bloom filter: Example
  - Filter lookup:
    - We want to know if some element x was seen before
    - Compute h(x) for each hash function h
    - If all the resulting bit positions are 1
      - We have seen x before
      - Collisions: expect some number of false positives
    - If at least one of these positions is 0
      - We have not seen x before

Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Bloom filter: Performance
  - Probability of *false positive*
  - Depends on:
    - Density of 1's in the array (d_1)
    - Number of hash functions (k)

$$d_1^k$$

  - d_1=?
  - max(d_1) = num of inserted elements * k
  - In practice
    - collisions lower d_1
    - d_1 < max(d_1)

Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Bloom filter: Performance
  - Model: throwing darts
  - **d** darts = number of elements * number of hash functions
  - **t** targets = number of bits in the bloom filter
  - d_1 = Number of targets hit by at least one dart

Filtering streams

- Bloom filter: Performance
  - Model: throwing darts
  - Probability of given target is hit by a given (one) dart
    - 1/t
  - Probability of one dart not hitting the given target
    - 1 – 1/t
  - Probability none of d darts hit a given target is
    - $d_0 = (1 - 1/t)^d$

Copyright © 2022 CCL : Analysis of massive data sets

Filtering streams

- Bloom filter: Performance
  - Model: throwing darts
  - Probability none of d darts hit a given target (d_0)

$$(1 - 1/t)^d = (1 - 1/t)^{t(d/t)} \sim= e^{-d/t}$$

$$e^{-1}$$

Filtering streams

- Bloom filter: Performance
  - Model: throwing darts: Example
  - n = 1 billion, k = 5, m = 100 million
  - $t = 10^9$, $d = 5*10^8$
  - $d_0 = e^{-0.5} = 0.607$
  - $d_1 = 1 - d_0 = 0.393$ [$d_1 < 0.5$ (collisions)]
  - Probability of false positive
    - $d_1^k = 0.393^5 = 0.00937$ (<1%)

Copyright © 2022 CCL : Analysis of massive data sets

4. Counting distinct elements

Copyright © 2022 CCL : Analysis of massive data sets

Counting distinct elements

- Motivating example
  - Web site statistics
  - Number of unique users per month
  - Users are identified by IP addresses
  - 4 billion IP addresses
  - Obvious approach:
    - Keep list of all IP addresses with counts
    - Use efficient search structure (hash table, tree)

Copyright © 2022 CCL : Analysis of massive data sets

Counting distinct elements

- Problem
  - Maintaining a count of the number of distinct elements in the stream
  - Not enough space to store the set of counts
- Applications
  - Number of unique visitors
  - Number of distinct products sold
  - Number of different words in a crawled web page

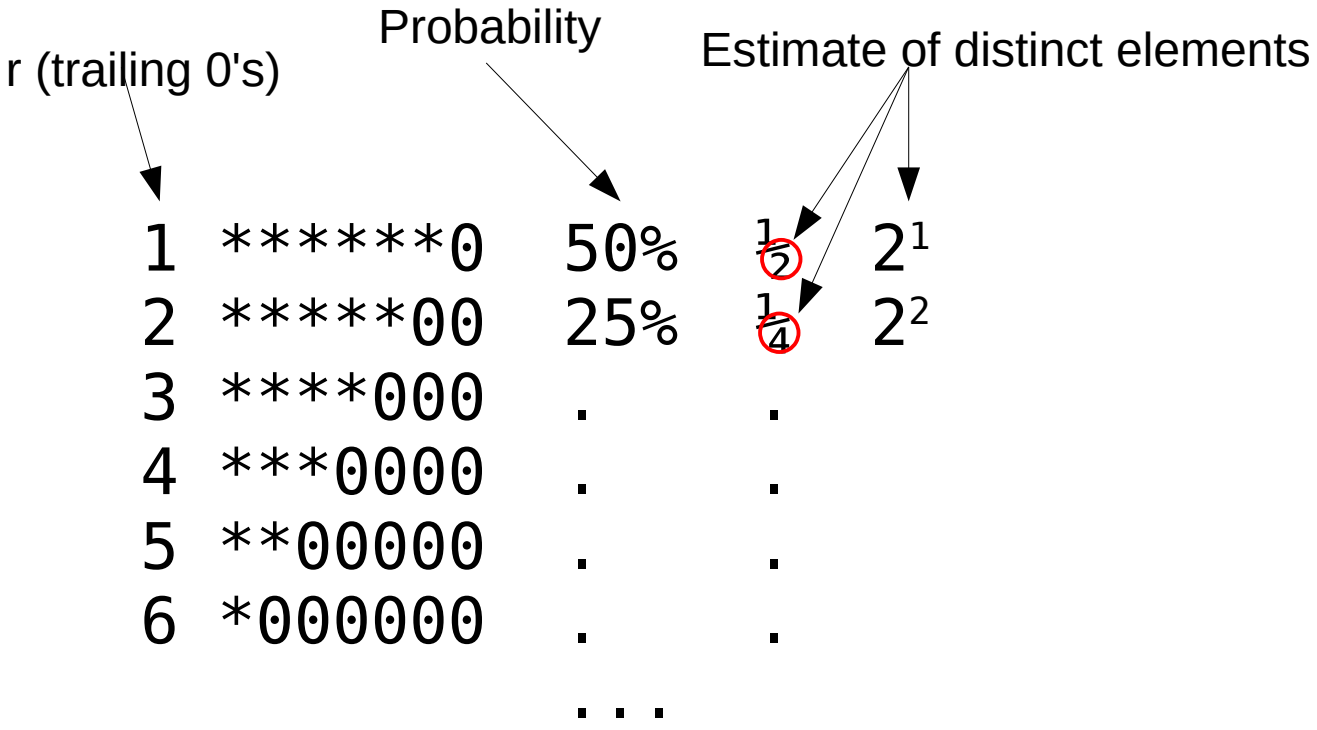
Counting distinct elements

- Flajolet-Martin algorithm
  - **n** = number of stream elements
  - **a** = stream element
  - Pick a hash function **h**
    - Maps stream elements to at least $\log_2 n$ bits
  - **r(a)** = number of trailing 0's in **h(a)**
  - **R** = **max(r(a))**, for all **a** in **S**
  - Distinct elements estimate:
    - 2^R

Copyright © 2022 CCL : Analysis of massive data sets

Counting distinct elements

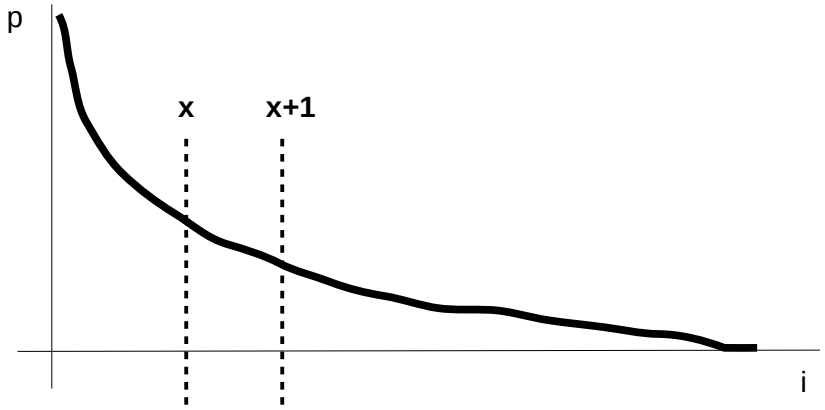
- Flajolet-Martin algorithm



Copyright © 2022 CCL : Analysis of massive data sets

Counting distinct elements

- Flajolet-Martin algorithm
  - R depends only on distinct elements
    - If same element appears in the stream it will have r(a) the same
  - Probability that r(a) = i
    - Goes down exponentially



  - when *i* is increased by 1
    - Need to double the number of elements to reach prob. of x+1
    - Thus, 2^R is good estimate

Copyright © 2022 CCL : Analysis of massive data sets

Counting distinct elements

- Flajolet-Martin algorithm
  - Formal analysis
    - probability that hash ends in at least **r** zeroes (p_1)
      - 2^{-r}
    - Probability of **not** seeing a tail of **r** zeroes among **m** elements (p_0)
      - $(1-2^{-r})^m$

$$p_0 = (1-2^{-r})^m = (1-2^{-r})^{2^r * m * 2^{-r}} \sim= e^{-m * 2^{-r}}$$

Counting distinct elements

- Flajolet-Martin algorithm
  - Formal analysis
    - Probability of finding hash with **r** tail (p_1)
      - $m << 2^r$
        - $p_0 = e^{-m2^{-r}} \sim= 1 \rightarrow p_1 = 0$
      - $m >> 2^r$
        - $p_0 = e^{-m2^{-r}} \sim= 0 \rightarrow p_1 = 1$
    - Result
      - **2^R** is always around **m**!
        - Not too high, not too low

Counting distinct elements

- Flajolet-Martin algorithm
  - Problems
    - 2^R can be too large
  - Workaround
    - Use many hash functions h_i and obtain many samples R_i
    - Combine samples
      - Average: problem with large values
      - Median: all powers of 2
    - Solution
      - Partition samples into small groups
      - Count average in groups and take median of the averages